

LABSHEET-8**DISHA SAMANTA****CU24250161****ROLL NO-20****SEC-B****1. Import the required Python libraries and load the dataset into a PandasDataFrame.**

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import plot_tree, confusion_matrix, accuracy_score, precision_score, recall_score, f1_score, classification_report
```

2. Select a suitable dataset such as the Telco Customer Churn Dataset, BankCustomer Churn Dataset, or IBM HR Employee Attrition Dataset.

```
from google.colab import files
uploaded = files.upload()

import pandas as pd
df = pd.read_csv(next(iter(uploaded)))
df.head()
```

Choose files Bank Custo...rediction.csv

Bank Customer Churn Prediction.csv(text/csv) - 561600 bytes, last modified: 07/09/2026 - 100% done

Saving Bank Customer Churn Prediction.csv to Bank Customer Churn Prediction (1).

	customer_id	credit_score	country	gender	age	tenure	balance	products_r
0	15634602	619	France	Female	42	2	0.00	
1	15647311	608	Spain	Female	41	1	83807.86	
2	15619304	502	France	Female	42	8	159660.80	
3	15701354	699	France	Female	39	1	0.00	
4	15737888	850	S i F l		43	2	125510.82	

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

3. Perform data preprocessing by handling missing values, encoding categorical variables, and selecting relevant features for prediction.

```
print(df.isnull().sum())

df.drop('customer_id', axis=1, inplace=True)

le_country = LabelEncoder()
df['country'] = le_country.fit_transform(df['country'])

le_gender = LabelEncoder()
df['gender'] = le_gender.fit_transform(df['gender'])

df.head()
```

```
customer_id      0
credit_score      0
country          0
gender           0
age              0
tenure           0
balance          0
products_number  0
credit_card      0
active_member    0
estimated_salary 0
churn            0
dtype: int64
```

	credit_score	country	gender	age	tenure	balance	products_number	credit
0	619	0	0	42	2	0.00	1	
1	608	2	0	41	1	83807.86	1	
2	502	0	0	42	8	159660.80	3	
3	699	0	0	39	1	0.00	2	
4	850	2	0	43	2	125510.82	1	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

4. Split the dataset into training and testing sets using an appropriate train-test ratio (e.g. 80:20).

```
X = df.drop('churn', axis=1)
y = df['churn']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("Training set:", X_train.shape)
print("Testing set:", X_test.shape)
```

```
Training set: (8000, 10)
Testing set: (2000, 10)
```

5. Train a Decision Tree Classifier using the training dataset and configure suitable parameters such as maximum tree depth and splitting criteria (Gini Index or Entropy).

```
dt_model = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=5,
```

```

        min_samples_split=10,
        min_samples_leaf=5,
        random_state=42
    )
dt_model.fit(X_train, y_train)

print("Decision Tree model trained successfully!")
print("Tree depth:", dt_model.get_depth())
print("Number of leaves:", dt_model.get_n_leaves())

```

```

Decision Tree model trained successfully!
Tree depth: 5
Number of leaves: 27

```

6. Use the trained model to predict customer churn for the testing dataset.

```

y_pred = dt_model.predict(X_test)

y_pred_proba = dt_model.predict_proba(X_test)[:, 1]

results_df = pd.DataFrame({
    'Actual': y_test.values[:10],
    'Predicted': y_pred[:10],
    'Churn_Probability': y_pred_proba[:10].round(3)
})
print("Sample Predictions vs Actual Values:\n")
print(results_df)

print("\nPredicted Class Distribution:")
print(pd.Series(y_pred).value_counts())

```

Sample Predictions vs Actual Values:

	Actual	Predicted	Churn_Probability
0	0	0	0.008
1	0	0	0.048
2	0	0	0.037
3	0	0	0.048
4	0	0	0.183
5	0	0	0.183
6	0	0	0.048
7	0	0	0.183
8	0	1	0.590
9	0	0	0.037

Predicted Class Distribution:

```

0    1752
1     248
Name: count, dtype: int64

```

7. Evaluate the model using a Confusion Matrix, Accuracy, Precision, Recall, and F1-Score.

```
cm = confusion_matrix(y_test, y_pred)
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Confusion Matrix:\n", cm)
print(f"\nAccuracy : {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall    : {rec:.4f}")
print(f"F1-Score : {f1:.4f}")
print("\nClassification Report:\n", classification_report(y_test, y_pred))

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['No Churn', 'Churn'],
            yticklabels=['No Churn', 'Churn'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

Confusion Matrix:

```
[[1530  63]
 [ 222 185]]
```

Accuracy : 0.8575

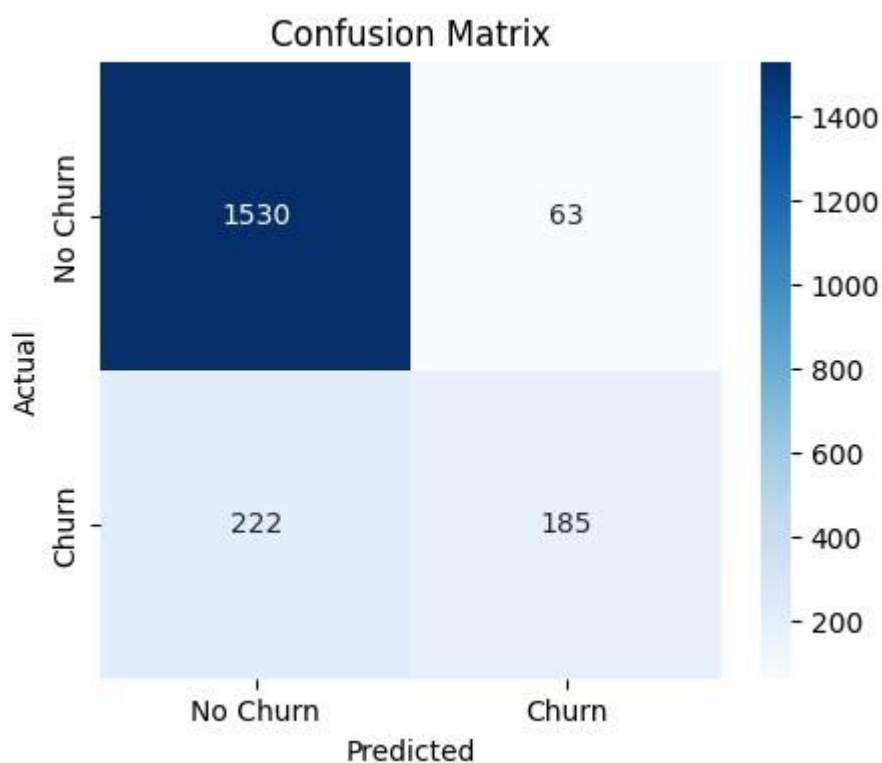
Precision: 0.7460

Recall : 0.4545

F1-Score : 0.5649

Classification Report:

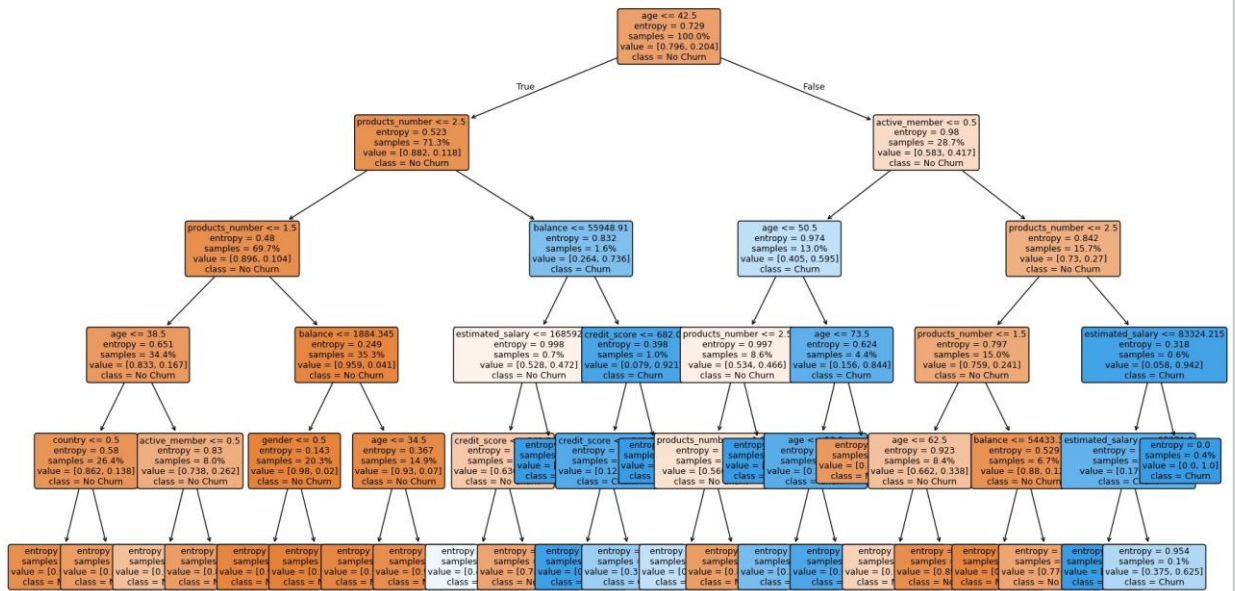
	precision	recall	f1-score	support
0	0.87	0.96	0.91	1593
1	0.75	0.45	0.56	407
accuracy			0.86	2000
macro avg	0.81	0.71	0.74	2000
weighted avg	0.85	0.86	0.84	2000



8. Visualize the Decision Tree to understand the decision-making process and identify the most influential features.

```
plt.figure(figsize=(22, 12))
plot_tree(dt_model,
          feature_names=X.columns,
          class_names=['No Churn', 'Churn'],
          filled=True,
          rounded=True,
          proportion=True,
```


Decision Tree for Customer Churn Prediction



```

|--- age <= 42.50
|   |--- products_number <= 2.50
|       |--- products_number <= 1.50
|           |--- age <= 38.50
|               |--- country <= 0.50
|                   |--- class: 0
|               |--- country > 0.50
|                   |--- class: 0
|           |--- age > 38.50
|               |--- active_member <= 0.50
|                   |--- class: 0
|               |--- active_member > 0.50
|                   |--- class: 0
|           |--- products_number > 1.50
|               |--- balance <= 1884.34
|                   |--- gender <= 0.50
|                       |--- class: 0
|                   |--- gender > 0.50
|                       |--- class: 0
|               |--- balance > 1884.34
|                   |--- age <= 34.50
|                       |--- class: 0
|                   |--- age > 34.50
|                       |--- class: 0
|           |--- products_number > 2.50
|               |--- balance <= 55948.91
|                   |--- estimated_salary <= 168592.80
|                       |--- credit_score <= 649.50
|                           |--- class: 1
|                       |--- credit_score > 649.50
|                           |--- class: 0
|                   |--- estimated_salary > 168592.80
|                       |--- class: 1
|               |--- balance > 55948.91
|                   |--- credit_score <= 682.00
|                       |--- credit_score <= 640.50
|                           |--- class: 1
  
```

```
| | | | | |--- credit_score > 640.50
| | | | | |--- class: 1
| | | | | |--- credit_score > 682.00
| | | | | |--- class: 1
|--- age > 42.50
| |--- active_member <= 0.50
| | |--- age <= 50.50
| | | |--- products_number <= 2.50
| | | | |--- products_number <= 1.50
| | | | |--- class: 1
| | | | |--- products_number > 1.50
| | | | |--- class: 0
| | | |--- products_number > 2.50
| | | |--- class: 1
| | |--- age > 50.50
| | | |--- age <= 73.50
| | | | |--- age <= 53.50
| | | | |--- class: 1
| | | | |--- age > 53.50
| | | | |--- class: 1
| | | |--- age > 73.50
| | | |--- class: 0
| |--- active_member > 0.50
| | |--- products_number <= 2.50
| | | |--- products_number <= 1.50
| | | |--- age <= 62.50
| | | | |--- class: 0
| | | |--- age > 62.50
| | | |--- class: 0
```